

```

# <Pulkit Saxena>                <20 October 2023>
# <Assignment 4>
# <Description>: In this assignment we will learn about how to make a tic-tac-toe
game

# CITATION: Much of the code in this assignment is heavily modified/ adapted from
code originally created by ChatGPT
# CITATION: ACCESSED: 3-15-2023
# CITATION: URL: https://chat.openai.com

import random
import sys

def printTitleMaterial():
    """Prints the title material for the game, including the student's name, class,
    and section number.
    """
    print("Four In Sequence!")
    print()

    print("By: <Pulkit Saxena>")
    print("[COM S 127 <J>]")
    print()

def initialChoice():
    """Allows the user to choose whether to [p]lay, get [i]nstructions, or [q]uit.

    :return string: choice - A string containing either 'p', 'i', or 'q'.
    """
    choice = input("Choice? [p]lay, [i]nstructions, [q]uit: ")
    while choice != "p" and choice != "i" and choice != "q":
        print("ERROR: Please enter 'p', 'i', or 'q'...")
        choice = input("Choice? [p]lay, [i]nstructions, [q]uit: ")
    return choice

def chooseNumPlayers():
    """Allows the user to choose whether to play a game with [1] or [2] players.

    :return int: numPlayers - An integer, limited to strictly 1 or 2, to indicate
    the number of players in the game.
    """
    numPlayers = int(input("Number of Players? [1] / [2]: "))
    while numPlayers != 1 and numPlayers != 2:
        print("ERROR: Please enter either 1 or 2...")
        numPlayers = int(input("Number of Players? [1] / [2]: "))
    return numPlayers

def printBanner():
    """Prints out a nice header to delineate the game from the previous text
    output.
    """

print("#####")
print()
print("-- Starting New Round --")
print()

def getPlayerPiece(playerNumber):

```

"""Returns a string corresponding to the 'player' under consideration. Player 0 corresponds to an 'empty' square.
Player 1 corresponds to the 'X' pieces. Player 2 corresponds to the 'O' pieces.

:param int playerNumber: The 'player' whose piece we wish to know.
:return string: piece - A string containing either '.', 'X', or 'O' for player 0 (empty), 1, or 2, respectively.

```
"""
piece = ""
if playerNumber == 0:
    piece = "."
elif playerNumber == 1:
    piece = "X"
elif playerNumber == 2:
    piece = "O"
return piece
```

def createBoard(width, height):
 """Creates the underlying data structure for the game - a list of lists. This function also sets all the 'spaces' in the 'gameboard' to be 'empty' (player 0) spaces.

An example 6x7 gameboard (7 width, 6 height) would be created/ indexed into as follows:

```
# 0.....
# 1.....
# 2.....
# 3.....
# 4.....
# 5.....
# 0123456
```

Each 'sub-list' in the outer list represents a 'row' of the board. Each entry in a 'row' represents the 'column' space at that position for that row.

:param int width: How many 'spaces' wide to make the gameboard.
:param int height: How many 'spaces' high to make the gameboard.
:return list of lists: board - The data structure that contains the contents of the gameboard. Only contains 'Player 0' pieces by default.

```
"""
empty = getPlayerPiece(0)
board = []
for i in range(0, height):
    board.append([])
    for j in range(0, width):
        board[i].append(empty)
return board
```

def printBoard(board):
 """Prints out the gameboard to the screen - including a row of digits at the bottom which correspond to columns the players can choose.

:param list of lists board: The data structure that contains the contents of the gameboard.

```
"""
for row in board:
    for column in row:
        print(column, end="")
```

```

        print()
    for i in range(0, len(board[0])):
        print(i, end=" ")
    print()
    print()

def getColumnInt(board, playerNumber):
    """Take in user input as a string, and convert it to an integer. This function
    constructs a prompt based on the
    playerNumber, and the number of columns on the gameboard.

    NOTE: This function does not apply any filtering or input validation of any
    kind - it just gets the number from the user.

    :param list of lists board: The data structure that contains the contents of
    the gameboard.
    :param int playerNumber: The player number to display on the text output.
    :return int: The number that the user entered.
    """

    return int(input("Player {0}, please select a column between (0-{1}):
    ".format(playerNumber, len(board[0]) - 1)))

def getInputInRange(board, playerNumber):
    """Prompt the user to enter an integer between 0 and the number of columns on
    the board minus one.
    This function will enforce this range, and will not allow values outside of it.

    :param list of lists board: The data structure that contains the contents of
    the gameboard.
    :param int playerNumber: The player number to display on the text output.
    :return int: col - The column the player wants to drop a piece inside.
    """

    col = getColumnInt(board, playerNumber)
    while col < 0 or col > len(board[0]) - 1:
        print("ERROR: Value must be between (0-{0}). You entered:
        {1}".format(len(board[0]) - 1, col))
        col = getColumnInt(board, playerNumber)
    return col

def getHumanInput(board, playerNumber):
    """This function collects input from a player corresponding to the column they
    want to drop a piece into.
    It enforces a range of columns between 0 and the number of columns on the board
    minus one by way of the getInputInRange() function.
    It also ensures that a column has at least one empty space to drop a piece in
    with the getOpenRow() function.

    :param list of lists board: The data structure that contains the contents of
    the gameboard.
    :param int playerNumber: The player number to display on the text output. It
    will be passed into the getInputInRange() function.
    :return int: col - The column the player wants to drop a piece inside.
    """

    col = getInputInRange(board, playerNumber)
    while getOpenRow(board, col) == -1:
        print("ERROR: Column {0} is full! Please choose a different
        column...".format(col))
        col = getInputInRange(board, playerNumber)

```

```

return col

def checkForNextMoveWin(board, playerNumber):
    """This function iterates through all the columns on the board and checks if
    each column has an open row.
    If the column has an open row, the function checks to see if placing a piece in
    this column (and thus in
    the open row) will result in a 'win condition' being present.

    If there is a 'win condition' present, this function will immediately terminate
    its execution, and return
    the column where placing a piece resulted in the win.

    Please note: This function does not make permanent changes to the gameboard. It
    will always revert whatever
    'test piece' it places in the board back to an 'empty' piece.

    :param list of lists board: The data structure that contains the contents of
    the gameboard.
    :param int playerNumber: The player number whose piece this function is to test
    for a 'win condition.'
    :return int: col - The column where the 'win condition' was found. It returns -
    1 if there is no 'win condition.'
    """
    empty = getPlayerPiece(0)
    piece = getPlayerPiece(playerNumber)

    # TODO: Write code in this function that accomplishes the specification
    outlined in the documentation comment above.
    # (KEEP this TODO line when completed) (1 pt.)
    # HINT: Each comment below represents a single line of code at the appropriate
    indentation level.
    # HINT: Do not start with this function - start with the checkWinner() function
    and get that working in 2 player mode.
    # NOTE: You do not need to do this according to the comments below - you may do
    this however you wish.

    for column in range(0, len(board[0])):

        # use getOpenRow() to check if the column has an open row, and return the row
        number if there is one
        row = getOpenRow(board, column)

        # if the column has an open row, meaning getOpenRow() did not return -1,
        then perform several logical steps
        if row != -1:
            # set the board, at the location [row][col] to be the player piece
            found above with the placePiece() function
            placePiece(board, row, column, piece)
            # use checkWinner() to see if there is a winning move that resulted
            from changing the board on the line above
            winningMove = checkWinner(board, playerNumber)
            # if there was a winning move found, reset the board, at the
            location [row][col] to be the empty piece found above with the placePiece()
            function
            if winningMove != False:
                placePiece(board, row, column, empty)
                # if there was a winning move found, immediately return the current

```

```

column number
        return column
    # reset the board, at the location [row][col] to be the empty piece
    found above with the placePiece() function
    placePiece(board, row, column, empty)

    # return -1, as there was no winning move found
    return -1

def checkAdjacent(board, playerNumber):
    """This function aids the computer in choosing which column to drop a piece
    into. It iterates through all available columns, finds
    an available row, and considers all the surrounding pieces relative to that
    space on the gameboard. If it finds an adjacent piece
    belonging to the player, the current column is added to a list called
    'adjacents' as a candidate to have a piece dropped into.
    If multiple pieces surround the space on the gameboard, the column will be
    added multiple times to the 'adjacents' list. Finally,
    after all of the columns have been analyzed, a random entry in the 'adjacents'
    list is chosen as the column for the computer to
    drop a piece into.

    This function has the benefit of being quite simple, but will create 'runs' of
    pieces which 'connect' to one another. This is
    behavior that would be expected in this type of game. It will make the game
    appear that it is being played by something more
    advanced than a simple random number generator.

    Furthermore, the more pieces which are adjacent to the candidate board space,
    the greater the probability that this space will
    be selected. However - all the entries in the 'adjacents' list have an equal
    chance of being picked, so it is not a 'given'
    that a certain column will be chosen.

    NOTE: This portion of the AI algorithm is (mostly) the original thought/
    creation of Matthew Holman - *NOT* ChatGPT. However, the initial
    column/ row looping scheme was taken from ChatGPT generated code.

    :param list of lists board: The data structure that contains the contents of
    the gameboard.
    :param int playerNumber: The player number whose piece this function is to test
    for adjacent pieces.
    :return int: col - The value that is randomly chosen from the 'adjacents' list.
    It returns -1 if the 'adjacents' list has < 2 entries.
    """
    col = -1
    piece = getPlayerPiece(playerNumber)
    adjacents = []

    # TODO: Write code in this function that accomplishes the specification
    outlined in the documentation comment above.
    # (KEEP this TODO line when completed) (1 pt.)
    # HINT: Each comment below represents a single line of code at the appropriate
    indentation level.
    # HINT: Do not start with this function - start with the checkWinner() function
    and get that working in 2 player mode.
    # HINT: You have been given the code to check the first adjacent piece - you
    will need to check all the other adjacent pieces.
    # NOTE: You do not need to do this according to the comments below - you may do

```

this however you wish.

```
# HINT: use this diagram to help with debugging
# 0.....
# 1.....
# 2.....
# 3.....
# 4.....
# 5.....
# 0123456

for column in range(0, len(board[0])):
    row = getOpenRow(board, column)
    if row != -1:
        # upper left piece (up one row, left one column)
        #
        # do a check to ensure we don't get an 'index out of range' error
        if row - 1 >= 0 and column - 1 >= 0:
            # do a check to see if the board location under consideration is
equal to the piece found above
            if board[row-1][column-1] == piece:
                # append the column under consideration to the 'adjacents' list
                adjacents.append(column)

        for column in range(0, len(board[0])):
            row = getOpenRow(board, column)
            if row != -1:
                # upper left piece (up one row, left one column)
                #
                # do a check to ensure we don't get an 'index out of range' error
                if row - 1 >= 0 and column - 1 >= 0:
                    # do a check to see if the board location under consideration is
equal to the piece found above
                    if board[row-1][column-1] == piece:
                        # append the column under consideration to the 'adjacents' list
                        adjacents.append(column)

    for column in range(0, len(board[0])):
        row = getOpenRow(board, column)
        if row != -1:
            # left piece (left one column)

            # ensure we don't get an 'index out of range' error
            if row - 1 >= 0 and column - 1 >= 0:
                # do a check to see if the board location under consideration is
equal to the piece found above
                if board[row][column-1] == piece:
                    # append the column under consideration to the 'adjacents' list
                    adjacents.append(column)

    for column in range(0, len(board[0])):
        row = getOpenRow(board, column)
        if row != -1:
            # lower left piece (down one row, left one column)
            #
            # ensure we don't get an 'index out of range' error
            if row - 1 >= 0 and column - 1 >= 0:
                # do a check to see if the board location under consideration is
equal to the piece found above
```

```

        if board[row][column-1] == piece:
            # append the column under consideration to the 'adjacents' list
            adjacents.append(column)

    for column in range(0, len(board[0])):
        row = getOpenRow(board, column)
        if row != -1:
            # lower piece (down one row)
            #
            # ensure we don't get an 'index out of range' error
            if row - 1 >= 0 and column - 1 >= 0:
                # do a check to see if the board location under consideration is
equal to the piece found above
                if board[row][column] == piece:
                    # append the column under consideration to the 'adjacents' list
                    adjacents.append(column)

    for column in range(0, len(board[0])):
        row = getOpenRow(board, column)
        if row != -1:
            # lower right piece (down one row, right one column)
            #
            # ensure we don't get an 'index out of range' error
            if row - 1 >= 0 and column - 1 >= 0:
                # do a check to see if the board location under consideration is
equal to the piece found above
                if board[row][column] == piece:
                    # append the column under consideration to the 'adjacents' list
                    adjacents.append(column)

    for column in range(0, len(board[0])):
        row = getOpenRow(board, column)
        if row != -1:
            # right piece (right one column)
            #
            # ensure we don't get an 'index out of range' error
            if row - 1 >= 0 and column - 1 >= 0:
                # do a check to see if the board location under consideration is
equal to the piece found above
                if board[row][column] == piece:
                    # append the column under consideration to the 'adjacents' list
                    adjacents.append(column)

    for column in range(0, len(board[0])):
        row = getOpenRow(board, column)
        if row != -1:
            # upper right piece (up one row, right one column)
            #
            # ensure we don't get an 'index out of range' error
            if row - 1 >= 0 and column - 1 >= 0:
                # do a check to see if the board location under consideration is
equal to the piece found above
                if board[row-1][column] == piece:
                    # append the column under consideration to the 'adjacents' list
                    adjacents.append(column)

    if len(adjacents) > 1:
        randVal = random.randrange(0, len(adjacents))
        col = adjacents[randVal]

```

```

        # print("piece:", piece, "adjacents:", adjacents, "randVal:", randVal,
"col:", col) # for debugging

    return col

def getComputerInput(board, currentPlayerTurn):
    """This is the 'AI'/ brain/ decision making structure the computer uses in the
single player game. The final decision comes after
    several different 'phases' in the computer choosing what to do. These phases
are governed by the content of the 'col' variable. If
    'col == -1', then, clearly, one of the phases has failed, and the next phase
should procede. 'col' will remain -1 until one of the
    phases succeeds. These phases are as follows:

    Firstly: If there is a winning move, the computer must take it no matter what.
    Secondly: If there is no winning move, but the opponent has a winning move, the
computer must block it no matter what.
    Thirdly: If the first two steps fail, check all the columns for adjacent pieces
and pick one where pieces would connect to one another.
    Finally: If the third step fails, meaning that there are not any moves that
would result in at least two pieces connecting, then
        just pick a random column and end the turn.

    :param list of lists board: The data structure that contains the contents of
the gameboard.
    :param int currentPlayerTurn: The player number for whom its turn it is.
    :return int: col - The column where the computer will place its piece.
    """
    opponentPlayerTurn = switchTurns(currentPlayerTurn)

    # check if there is a winning move - if there is then take it
    col = checkForNextMoveWin(board, currentPlayerTurn)
    # print("WIN")

    # check if the opponent has a winning move - if there is then block it
    if col == -1:
        col = checkForNextMoveWin(board, opponentPlayerTurn)
        # print("BLOCK")

    # check if there are any adjacent pieces available - try to connect to them
    if col == -1:
        col = checkAdjacent(board, currentPlayerTurn)
        # print("ADJACENT")

    # if no winning move is available, and no block is avaialable, and no adjacent
pieces are available - choose a random column
    if col == -1:
        col = random.randrange(0, len(board[0]))
        while getOpenRow(board, col) == -1:
            col = random.randrange(0, len(board[0]))
        # print("RANDOM")

    # print out a string stating the column that was chosen
    print("Player {0}, please select a column between (0-{1}):
{2}".format(currentPlayerTurn, len(board[0]) - 1, col))

    return col

def getOpenRow(board, col):

```


"""Iterates through all the rows of a given column (col), from bottom to top in the gameboard, and returns the first open row it finds.

:param list of lists board: The data structure that contains the contents of the gameboard.
:param int col: The column to check.
:return int: row - The row index of the first empty row from the bottom of the gameboard. It returns -1 if no empty row is found.
"""

```
empty = getPlayerPiece(0)
for row in range(len(board) - 1, -1, -1):
    if board[row][col] == empty:
        return row
return -1
```

def placePiece(board, row, col, piece):

"""Inserts a piece into the gameboard at a specific position.

:param list of lists board: The data structure that contains the contents of the gameboard.
:param int row: The row on the gameboard to insert the piece into.
:param int col: The column on the gameboard to insert the piece into.
:param string piece: The actual piece (should be "X", "O", or ".") to place into the gameboard
"""

```
board[row][col] = piece
```

def dropPieceIntoBoard(board, col, playerNumber):

"""Inserts a piece into the gameboard in a given column. This is the function that should be called once either a human or computer player determines which column to drop their piece into. It finds the lowest available row on the gameboard, given the specified column, and places the appropriate piece at that location.

Before this function is called, it will have previously been determined whether the column (col) is valid or not.
Therefore, there is no need to worry about that here.

:param list of lists board: The data structure that contains the contents of the gameboard.
:param int col: The column on the gameboard to insert the piece into.
:param int playerNumber: The player number whose piece this function is to 'drop' into the board.
"""

```
row = getOpenRow(board, col)
placePiece(board, row, col, getPlayerPiece(playerNumber))
```

def checkDraw(board):

"""This function checks to see if all the spaces on the board have been filled up. If they have been, then there has likely been a 'draw' as there are no possible additional moves available for any player.

:param list of lists board: The data structure that contains the contents of the gameboard.
:return boolean: Returns False immediately if it finds a single empty space on the board - meaning there cannot be a draw. It returns True otherwise.
"""

```
#empty = getPlayerPiece(0)
```

```

# TODO: Write code in this function that accomplishes the specification
outlined in the documentation comment above.
# (KEEP this TODO line when completed) (1 pt.)
# HINT: Each comment below represents a single line of code at the appropriate
indentation level.
# HINT: Do not start with this function - start with the checkWinner() function
and get that working in 2 player mode.
# NOTE: You do not need to do this according to the comments below - you may do
this however you wish.

```

```

# iterate through all the rows in the board

```

```

# iterate through all the columns in a row

```

```

# check if the column is empty

```

```

# if a column is empty for a particular row, then no draw is
possible - return False immediately

```

```

# if there was not a single empty column found on the entire board, the board
must be entirely filled - return True for a draw

```

```

for row in range(0, len(board)):

```

```

    #iterate through columns

```

```

    for col in range(len(board)-1):

```

```

        # check if the column is empty

```

```

        if col == 0:

```

```

            return False

```

```

if checkWinner(board, 1) or checkWinner(board, 2):

```

```

    return False

```

```

return True

```

```

def checkWinner(board, playerNumber):

```

```

    """This function checks the gameboard to see if a winning condition is present.
Meaning, if a given piece (determined by the
    playerNumber variable) occurs four (4) times in a row, column, positively
sloped diagonal, or negatively sloped diagonal.

```

```

    These checks should occur one at a time, and should be done by considering a
'window' for each pattern of length 4. Meaning,
    if checking for a horizontal win, then it does not make sense to start a check
beyond column index 3 (assuming the board is 7
    columns wide). The reason for this is because, if such a check started at index
5, for example, then the board would 'run out
    of spaces' before 4 pieces could even be inserted!

```

```

    As there are 4 different 'winning condition' configurations there are,
therefore, 4 different 'windows' to be considered. Each
    'window' will be different from all the other 'windows,' and should accommodate
all possible 'winning condition' configurations
    of that type. For example, assuming a 6 row x 7 col gameboard, there should be
a window that accommodates a positively sloped
    diagonal from (3, 3) to (0, 6). This diagonal would include gameboard spaces
(3, 3), (2, 4), (1, 5), and (0, 6). The 'check' for
    this configuration should likely start at (3, 3) - placing the 'window' in the
bottom-left of the gameboard.

```

```

:param list of lists board: The data structure that contains the contents of

```

the gameboard.

```
:param int playerNumber: The player number whose piece this function checks to  
see if there is a winning condition.  
:return boolean: Returns True if a winning condition is found. It returns False  
if a winning condition is not found.  
"""
```

```
# TODO: Write code in this function that accomplishes the specification  
outlined in the documentation comment above.
```

```
# (KEEP this TODO line when completed) (1 pt.)
```

```
# HINT: Each comment below represents multiple lines of code. However, each  
code block should closely resemble the first.
```

```
# HINT: Start with the this function and get it working in 2 player mode.
```

```
# HINT: You have been given the code to check the first winning condition - you  
will need to check all the other winning conditions.
```

```
# NOTE: You do not need to do this according to the comments below - you may do  
this however you wish.
```

```
# HINT: use this diagram to help with debugging
```

```
# 0.....
```

```
# 1.....
```

```
# 2.....
```

```
# 3.....
```

```
# 4.....
```

```
# 5.....
```

```
# 0123456
```

```
# get the piece to check for
```

```
piece = getPlayerPiece(playerNumber)
```

```
# check horizontal locations
```

```
for row in range(0, len(board)):
```

```
    for column in range(0, len(board[0]) - 3):
```

```
        if board[row][column] == piece and board[row][column+1] == piece and  
board[row][column+2] == piece and board[row][column+3] == piece:  
            return True
```

```
# check vertical locations
```

```
for row in range(0, len(board) - 3):
```

```
    for column in range(0, len(board[0])):
```

```
        if board[row][column] == piece and board[row+1][column] == piece and  
board[row+2][column] == piece and board[row+3][column] == piece:  
            return True
```

```
# check negatively sloped diagonals
```

```
for row in range(0, len(board)):
```

```
    for column in range(0, len(board[0]) - 3):
```

```
        if board[row][column] == piece and board[row-1][column-1] == piece and  
board[row-2][column-2] == piece and board[row-3][column-3] == piece:  
            return True
```

```
# check positively sloped diagonals
```

```
for row in range(0, len(board) - 3):
```

```
    for column in range(0, len(board[0]) - 3):
```

```
        if board[row][column] == piece and board[row+1][column+1] == piece and  
board[row+2][column+2] == piece and board[row+3][column+3] == piece:  
            return True
```

```
    return False
```

```
def resetGameOptions():
```

```
    """Calling this function will reset all the relevant gameplay variables to  
their pre-gameplay state so a new game can begin.
```

```
    :return int, boolean, boolean: These positional return values will set  
'currentPlayerTurn' to 1 and the 'winner' and 'draw' variables to False.  
    """
```

```
    currentPlayerTurn = 1  
    winner = False  
    draw = False  
    return currentPlayerTurn, winner, draw
```

```
def switchTurns(currentPlayerTurn):
```

```
    """Change the current turn from Player 1 to Player 2 and vice versa.
```

```
    :param int currentPlayerTurn: _description_  
    :return int: If currentPlayerTurn was 1, it changes to 2. If currentPlayerTurn  
was 2, it changes to 1.  
    """
```

```
    return ((currentPlayerTurn % 2) + 1)
```

```
def main():
```

```
    """The main function for the game. The primary gameplay loop is located here.  
    """
```

```
    # main script running control variable  
    running = True
```

```
    # gameplay variables  
    currentPlayerTurn = 1 # can be 1 or 2  
    winner = False  
    draw = False
```

```
    # print the title/ author information  
    printTitleMaterial()
```

```
    # play the game
```

```
    while running:
```

```
        choice = initialChoice()  
        if choice == "p":
```

```
            # reset relevant gameplay variables  
            currentPlayerTurn, winner, draw = resetGameOptions()
```

```
            # choose number of players  
            numPlayers = chooseNumPlayers()
```

```
            # create gameboard  
            board = createBoard(7, 6)
```

```
            # round setup
```

```

printBanner()

# print board
printBoard(board)

# main game loop
while True:

    # get input for dropping a piece inside the board (switch between
    player 1 and player 2)
    if numPlayers == 1:
        if currentPlayerTurn == 1:
            col = getHumanInput(board, currentPlayerTurn)
        elif currentPlayerTurn == 2:
            col = getComputerInput(board, currentPlayerTurn)
        else:
            print("ERROR: currentPlayerTurn is neither 1 or 2! It is
actually: {0}".format(currentPlayerTurn))
            sys.exit()
    elif numPlayers == 2:
        col = getHumanInput(board, currentPlayerTurn)
    else:
        print("ERROR: numPlayers is neither 1 or 2! It is actually:
{0}".format(numPlayers))
        sys.exit()

    # update the board with the new piece inside of it
    dropPieceIntoBoard(board, col, currentPlayerTurn)

    # print board
    printBoard(board)

    # check for winner
    winner = checkWinner(board, currentPlayerTurn)

    # check for a draw (all columns filled up completely)
    draw = checkDraw(board)

    # check of the game is over - if it is, break out of the gameplay
    loop
    if winner == True:
        print("-- Player {0} ({1}) Wins! --".format(currentPlayerTurn,
getPlayerPiece(currentPlayerTurn)))
        print()
        break
    elif draw == True:
        print("-- Draw! --")
        print()
        break
    else: # if the game is not over, print that the turn is over
        print("-- End Of Player {0} ({1}) Turn
--".format(currentPlayerTurn, getPlayerPiece(currentPlayerTurn)))
        print()
        # switch turns
        currentPlayerTurn = switchTurns(currentPlayerTurn)

    elif choice == "i":
        print ("The user selects a game between one or two players.A one player
game has the user play against the computer.A two player game has two users play

```

against one another. Each player takes turns 'dropping' their 'pieces' into the board. The goal is for one player to connect four pieces in an unbroken sequence. A sequence can be horizontal, vertical, or diagonal. The game ends when one player has... FOUR IN SEQUENCE!")

```
    # TODO: Print out the instructions/ algorithm for the game.
    #         (KEEP this TODO line when completed) (1 pt.)
    # HINT: You can delete the 'pass' statement - it does nothing.
    pass
    elif choice == "q":
        print("Goodbye")
        # TODO: Set the 'running' variable to False, and print out a 'Goodbye!'
message to the player.
        #         (KEEP this TODO line when completed) (1 pt.)
        # HINT: You can delete the 'pass' statement - it does nothing.
        pass
    else:
        print("ERROR: Variable 'choice' should have been 'p', 'i', or 'q', but
instead was:", choice)
        quit()

if __name__ == "__main__":
    main()
```